

Prompt Engineering Assignment

1. Before TL;DR

Prompt : Explain about prompt engineering and its techniques

Prompt Response:

PROMPT ENGINEERING: A COMPREHENSIVE GUIDE

=====

WHAT IS PROMPT ENGINEERING?

Prompt engineering is the practice of designing, refining, and optimizing the inputs (prompts) given to large language models (LLMs) to elicit the most accurate, relevant, and useful outputs.

WHY IT MATTERS IN 2026

- Performance: Good prompts improve output quality without retraining models
- Cost-effectiveness: Faster and cheaper than fine-tuning for many use cases
- Safety & Control: Helps align AI behavior with ethical guidelines
- Accessibility: No deep technical expertise required

CORE TECHNIQUES

Foundational:

- Zero-shot: Direct instruction, no examples → Simple tasks
- One-shot: One example → Teaching tone/format
- Few-shot: Multiple examples → Complex patterns

Reasoning:

- Chain-of-Thought: "Think step by step" → Math, logic, troubleshooting
- Tree of Thoughts: Explore multiple paths → Strategic planning
- Self-Consistency: Aggregate multiple answers → Reduce errors

BEST PRACTICES

✅ DO:

- Be specific: "Summarize in 3 bullet points focusing on risks"
- Use delimiters: Separate instructions with ### or ---
- Provide context first, then task
- Iterate and test small changes
- Anchor outputs with pre-filled structures

❌ AVOID:

- Vague instructions ("Make it better")
- Contradictory constraints
- Assuming unstated context
- Overloading prompts without structure

THE CRISP FRAMEWORK

C - Context: Provide necessary background

R - Role: Assign a persona or expertise level

I - Instruction: State the task clearly and specifically

S - Structure: Define output format and constraints

P - Parameters: Add tone, length, audience guidance

EXAMPLE PROMPT:

[Context] Reviewing a fintech customer support transcript.

[Role] Act as a senior risk analyst.

[Instruction] Identify potential fraud indicators.

[Structure] Return JSON: {"risk_level": "", "flags": [], "recommendation": ""}

[Parameters] Professional tone. Under 150 words.

SECURITY & RESPONSIBLE PROMPTING

Defensive Strategies:

- Input sanitization: Evaluate user inputs before passing to model
- Output constraints: Explicitly forbid certain response types
- Scaffolding: Add safety-evaluation reasoning steps
- Red teaming: Test prompts with adversarial examples

Prompt : TL;DR Prompt Engineering and its techniques

Response:

PROMPT ENGINEERING: TL;DR Techniques Quick Reference

Designing clear, strategic inputs to get accurate, useful outputs from AI.

→ "Programming with natural language"

7 ESSENTIAL TECHNIQUES

1. ZERO/FEW-SHOT PROMPTING

- Zero-shot: Ask directly with no examples
- Few-shot: Provide 2-5 examples to show pattern
- ✓ Best for: Classification, formatting, simple Q&A

2. CHAIN-OF-THOUGHT (CoT)

- Add: "Let's think step by step" or "Show your reasoning"
- ✓ Best for: Math, logic, troubleshooting, complex decisions

3. ROLE PROMPTING

- Start with: "You are a [expert role]..."
- ✓ Best for: Domain-specific tone, perspective, or expertise

4. STRUCTURED OUTPUT

- Specify: JSON, XML, bullets, tables, word limits
- ✓ Best for: APIs, automation, consistent formatting

5. RETRIEVAL-AUGMENTED GENERATION (RAG)

- Ground responses in your documents/knowledge base
- ✓ Best for: Factual accuracy, company-specific info, up-to-date data

6. SELF-REFLECTION / REFINEMENT

- Ask AI to critique or improve its own answer
- ✓ Best for: High-stakes content, iterative quality improvement

7. REACT (REASON + ACT)

- Combine reasoning with tool use (search, code, APIs)
- ✓ Best for: Multi-step tasks requiring external actions

CRISP PROMPT TEMPLATE

[Context] : <Background info the AI needs>

[Role] : You are a <expertise/role>

[Instruction] : <Clear, specific task>

[Structure] : <Output format: JSON/bullets/table/word limit>

[Parameters] : Tone: ____ | Length: ____ | Audience: ____

EXAMPLE:

[Context] : Customer asked about refund policy for digital products.

[Role] : You are a friendly customer support specialist.

[Instruction] : Draft a polite response explaining our 14-day refund policy.

[Structure] : 3 short paragraphs, max 100 words total.

[Parameters] : Tone: Empathetic | Audience: Non-technical Customer

5 GOLDEN RULES

- ✓ Be specific → "3 bullet points on security risks" not "Tell me about risks"
- ✓ Set the role → "Act as a senior analyst..." improves relevance
- ✓ Define format → JSON, table, or bullets = easier to use outputs
- ✓ Context first → Background before task = better understanding
- ✓ Iterate → Small tweaks → big improvements

AVOID THESE MISTAKES

- X Vague prompts → vague answers
- X Contradictory instructions ("Be brief but detailed")
- X Assuming the AI knows your context or jargon
- X Overloading one prompt → break complex tasks into steps
- X Skipping security → always sanitize inputs and set Guardrails.

SECURITY QUICK TIPS

- Sanitize user inputs before passing to AI
- Explicitly forbid harmful or sensitive outputs
- Add "safety check" reasoning steps for high-risk tasks
- Test prompts with edge cases ("red teaming")

ONE-SENTENCE SUMMARY

"Tell the AI exactly what you want, how you want it, and in what format — then test and refine."

2. ELI15

Prompt : ELI15 Test Strategy Document
(Explain like I'm 15)

Response:

TEST STRATEGY DOCUMENT: THE ULTIMATE CHEAT SHEET

=====

🎯 For: Anyone who needs to test software without losing their mind

📖 Level: ELI15 (Simple, no jargon overload)

✨ Goal: Know what to test, how to test it, and when you're done

🤔 WHAT IS A TEST STRATEGY DOCUMENT? (THE SIMPLE VERSION)

=====

Imagine you're planning a HUGE video game launch 🎮

❌ Without a plan:

"Uh... let's just play it and see if it crashes?"

→ Bugs everywhere. Players mad. You stressed.

✅ With a Test Strategy:

"Here's EXACTLY what we'll test, who does what, what tools we use, and how we know it's ready."

→ Smooth launch. Happy players. You look like a pro.

👉 A Test Strategy Document = Your game plan for finding bugs BEFORE users do.

🧩 THE 7 KEY PARTS (WITH REAL EXAMPLES)

=====

1 WHAT ARE WE TESTING? (Scope)

✓ IN SCOPE:

- Login page & password reset
- Shopping cart & checkout
- Mobile app (iOS & Android)

✗ OUT OF SCOPE:

- Payment gateway (they handle it)
- Third-party ads
- Legacy desktop version

2 HOW WILL WE TEST? (Approach)

- Manual testing: Humans clicking through like real users
- Automated tests: Scripts that run 100x faster (for repetitive stuff)
- Beta testing: Let 100 real users try it before launch
- Stress testing: "What if 10,000 people sign up at once?"

3 WHO DOES WHAT? (Roles)

- 👤 QA Lead: Writes the plan, tracks progress
- 👥 Testers: Execute tests, report bugs
- 👨‍💻 Developers: Fix bugs, help with test data
- 👤 Product Owner: Says "yes, this is ready to launch"

4 WHAT TOOLS WILL WE USE? (Tech Stack)

- 🔧 Bug tracking: Jira, Trello, or GitHub Issues
- 🤖 Automation: Selenium, Cypress, or Playwright
- 📱 Mobile testing: BrowserStack or real devices
- 📊 Reporting: Simple spreadsheets or dashboards

5 WHAT DOES "DONE" LOOK LIKE? (Exit Criteria)

- 🎯 We're ready to launch when:
 - All HIGH-priority bugs are fixed ✓
 - 95% of test cases pass ✓
 - Performance: Page loads < 3 seconds ✓
 - Security scan: No critical vulnerabilities ✓
 - Product owner signs off ✓

6 WHAT COULD GO WRONG? (Risks & Mitigation)

- ⚠️ Risk: "What if the API is down during testing?"
 - 💡 Fix: Have a mock/stub version ready to use
- ⚠️ Risk: "What if we run out of time?"
 - 💡 Fix: Prioritize tests — test critical features first

7 TIMELINE: WHEN DOES STUFF HAPPEN? (Schedule)

- 📅 Week 1: Write test cases + set up tools
- 📅 Week 2: Manual testing (core features)
- 📅 Week 3: Automated tests + bug fixes
- 📅 Week 4: Final regression + sign-off 🚀

📝 SIMPLE TEMPLATE (COPY-PASTE READY)

TEST STRATEGY: [Project Name]

Version: 1.0 | Date: [Today] | Owner: [Your Name]

1. OBJECTIVE

"Ensure [app/feature] works correctly for [target users] before launch on [date]."

2. SCOPE

✅ Testing: [List features/pages/modules]

❌ Not testing: [List exclusions + why]

3. TEST APPROACH

- Manual testing for: [user flows, UI, edge cases]
- Automated testing for: [login, search, checkout]
- Beta testing with: [number] real users

4. ROLES & RESPONSIBILITIES

Person/Team	Responsibility	Contact
[Name]	Write & execute tests	@slack/email
[Name]	Fix bugs	@slack/email
[Name]	Approve launch	@slack/email

5. TOOLS & ENVIRONMENTS

- Bug tracker: [Jira/Trello/GitHub]
- Test env: [Staging URL + login info]
- Devices: [iOS 15+, Android 10+, Chrome, Safari]

6. TEST CASE EXAMPLES

- ◆ TC-001: User can log in with valid credentials
Steps: 1) Go to login 2) Enter email/pass 3) Click "Sign In"
Expected: Dashboard loads, welcome message shows
- ◆ TC-002: Cart updates when item quantity changes
Steps: 1) Add item 2) Change qty to 3 3) Refresh page

Expected: Total price updates, quantity saved

7. EXIT CRITERIA (When we're DONE)

- ☐ All critical bugs resolved
- ☐ 95%+ test cases passed
- ☐ Performance meets requirements
- ☐ Security scan clean
- ☐ Stakeholder sign-off received

8. RISKS & CONTINGENCIES

Risk	Likelihood	Impact	Mitigation
API delays	Medium	High	Use mock data
Tester unavailable	Low	Medium	Cross-train team
Scope creep	High	High	Freeze scope by [date]

9. APPROVALS

- ☒ QA Lead: _____ Date: __/__/__
- ☒ Dev Lead: _____ Date: __/__/__
- ☒ Product: _____ Date: __/__/__

=====

💡 PRO TIPS FOR TEENS (AND ADULTS WHO FEEL LIKE TEENS)

=====

✨ Start small: Don't try to test EVERYTHING. Focus on what matters most.

✨ Test like a user: Would YOUR little sibling understand this flow?

✨ Bugs are friends: Finding them early = saving stress later.

✨ Document as you go: Future-you will thank present-you.

✨ Communicate clearly: "Button broken" ❌ → "Checkout button doesn't respond on iOS Safari v16" ✅

✨ Automate the boring stuff: Let scripts handle repetitive clicks.

✨ Celebrate progress: Every bug fixed = one less fire at launch 🔥➡️✅

=====

🚫 COMMON MISTAKES (DON'T DO THESE)

- =====
- =====
- ❌ "We'll test everything!" → Too vague. Prioritize.
 - ❌ Waiting until the last minute → Bugs need TIME to fix.
 - ❌ Only testing the "happy path" → Test what happens when users do weird stuff.
 - ❌ Ignoring mobile → 60%+ of users are on phones.
 - ❌ No clear "done" criteria → Endless testing = missed deadlines.

=====

=====

🎯 ONE-PAGE SUMMARY (FOR YOUR BOSS/TEAM)

=====

=====

PROJECT: [Name]
LAUNCH DATE: [Date]

✅ WE WILL TEST:

- [Feature 1], [Feature 2], [Feature 3]
- Platforms: [Web, iOS, Android]

🔧 HOW:

- Manual + automated tests
- Beta group of [X] users
- Tools: [Jira, Selenium, BrowserStack]

🎯 WE'RE READY WHEN:

- Critical bugs = 0
- 95%+ tests pass
- Performance & security checks clear
- Product owner approves

🕒 TIMELINE:

Week 1: Prep → Week 2: Test → Week 3: Fix → Week 4: Launch 🚀

=====

=====

📖 HOW TO USE THIS DOCUMENT

=====

=====

1. COPY this entire template
2. PASTE into Word, Google Docs, or Notion
3. REPLACE [bracketed text] with your project details
4. SHARE with your team for feedback

5. UPDATE as you learn new things (it's a living doc!)

6. EXPORT as PDF when ready to present

✨ BONUS: Add emojis, colors, or your team's inside jokes to keep it engaging. Testing doesn't have to be boring!